HEALTHCARE KNOWLEDGE REASONING SYSTEM

Ardra Suresh,Jinsy Panamkattu, Neethu Joseph Chully, Reshma Mani

December 21, 2025

1 Introduction and Motivation

The healthcare sector requires accurate knowledge organization and reliable inference to facilitate efficient and secure decision-making processes. Medical knowledge is inherently complex, involving patients, diseases, symptoms, treatments, medications, and patient-specific constraints. In environments crucial for safety, like healthcare, traditional data-driven systems frequently fail to offer clear explanations and consistent outcomes, thereby compromising their reliability and trustworthiness. Symbolic Artificial Intelligence methods provide a viable solution by allowing for the explicit representation of domain knowledge and logically structured reasoning procedures.

This project introduces a Healthcare Disease & Treatment System that combines ontologies, knowledge graphs, and Answer Set Programming (ASP) within an interactive web-based environment. Ontologies precisely specify healthcare concepts and their interrelationships, whereas knowledge graphs meticulously store organized medical data in RDF format [1]. These semantic representations are automatically converted into ASP facts and then analyzed using the Clingo solver for performing non-monotonic reasoning, constraint checking, and treatment recommendations. A user-friendly interface crafted with HTML, CSS, and a Python Flask backend enables users to access patient information and examine explainable reasoning outcomes without technical hurdles. These elements showcase a blended symbolic structure that provides clear, logically sound, and user-friendly healthcare advice.

Project Scope

In Scope

- Ontology model for basic healthcare entities such as patient, disease, symptom, medicine, and allergy.

- Population of the knowledge graph with sample patient records and medical knowledge.
- Reasoning in an ASP-based framework to detect drug interactions and recommend appropriate treatments.
- Pre-application contradiction checks for patient allergies and existing medical conditions.

Out of Scope

- Integration with Electronic Health Record (EHR) systems.
- A full-scale medical ontology (the project is limited to a selected subset).
- Machine learning-based diagnosis prediction.
- Natural language processing of clinical notes.

2 Methodology

The project follows a symbolic AI pipeline combining ontology engineering, knowledge graph construction, and logic-based reasoning[2]. The following figure illustraties the system architecture which follows a structured, layered workflow. First, the OWL ontology is designed to formally represent domain concepts, relationships, and constraints. From this ontology, a knowledge graph is constructed, capturing entities and their interconnections as machine-interpretable facts. The knowledge graph is then converted into Answer Set Programming (ASP) rules and facts[3], enabling logic-based representation suitable for non-monotonic reasoning. These ASP representations are processed by a reasoning engine, which performs inference, consistency checking, and rule-based decision making to derive implicit knowledge. Finally, the results produced by the reasoning engine are presented through an intuitive user interface, allowing users to query the system, visualize inferred knowledge, and support informed decision-making.

2.1 Ontology and Knowledge Graph Design

The ontology was designed in Protégé using OWL 2 DL [4]. The main classes include:

- Patient - represents an individual receiving medical care.
- Disease - models medical conditions such as Diabetes, Hypertension, or Flu.

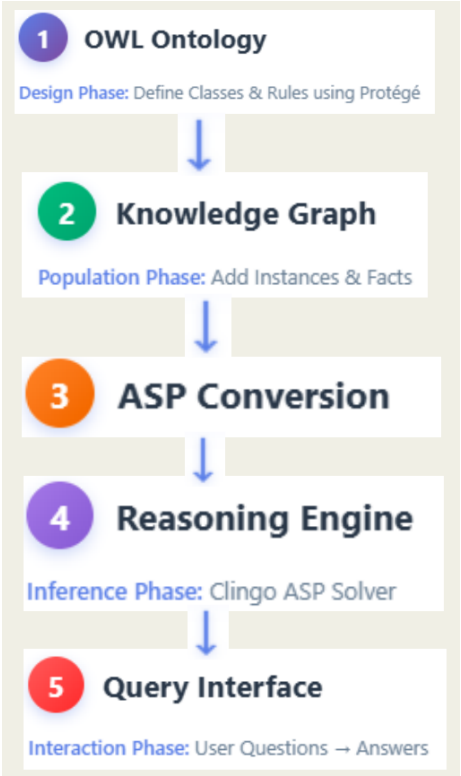

Figure 1: System Architecture of the Proposed Framework

- Symptom - captures observable indicators including Fever, Fatigue, and Headache.
- Treatment - represents therapeutic actions such as medication or therapy.
- Medication - denotes drugs prescribed as part of a treatment plan.

Key object properties were defined to describe relationships between these classes:

- `hasSymptom` (*Patient* → *Symptom*)
- `diagnosedWith` (*Patient* → *Disease*)
- `treatedBy` (*Disease* → *Treatment*)
- `usesMedication` (*Treatment* → *Medication*)

Data characteristics encompass elements like patient age and medication dosage. Additional constraints were implemented to maintain uniformity in modeling, such as requiring that a disease be linked to at least one therapy. The knowledge graph was populated manually with a small but representative dataset comprising examples of patients, diseases, symptoms, and treatments. This led to a lightweight KG containing dozens of triples, adequate to showcase reasoning abilities.

2.2 Answer Set Programming

The ASP component was implemented using Clingo[5]. ASP was chosen because it supports:

- Constraint-based reasoning
- Non-monotonic logic
- Multiple possible diagnoses (answer sets)

The main predicates include:

- `patient(P)`
- `symptom(P,S)`
- `disease(D)`
- `causes(D,S)`
- `possible_diagnosis(P,D)`
- `treatment(D,T)`

Rules infer possible diagnoses based on observed symptoms, while constraints eliminate inconsistent diagnoses. For instance, when a patient exhibits symptoms of fever and cough, the system might deduce that the condition could be influenza, provided other factors are not in conflict. The ASP module collaborates with the ontology and knowledge graph: the ontology specifies the domain vocabulary and organization, whereas ASP processes the extracted facts.

2.3 Hybrid Approach

The system demonstrates a hybrid symbolic architecture where:

- User queries are interpreted using ontology concepts.
- The knowledge graph provides structured medical knowledge.
- Answer Set Programming performs reasoning to infer diagnoses and treatments.

This hybrid method demonstrates how symbolic reasoning can enhance data-driven or language-model-based systems by offering explainability and logical assurances.

3 Evaluation

The developed system has a user-friendly interface . The following figure shows the interface.

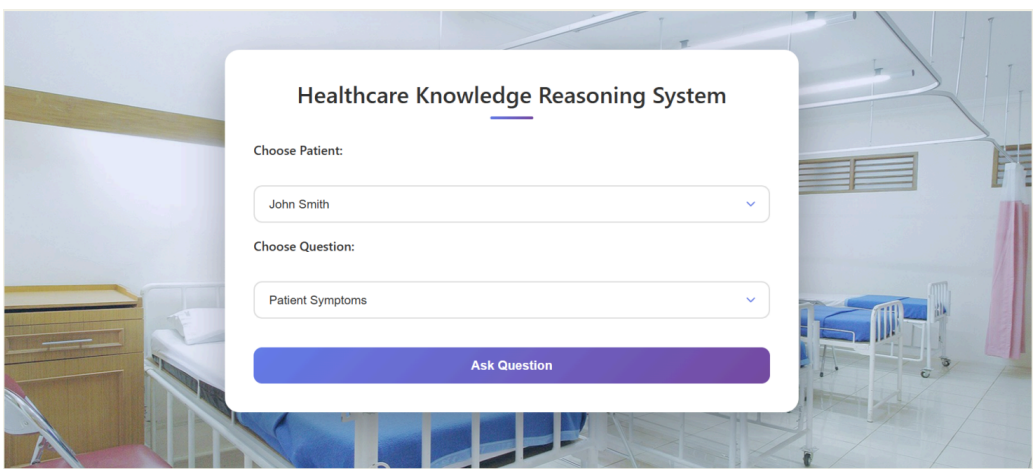

Figure 2: User Interface

- The system was evaluated by testing all major components through the web interface implemented in `index.html` and `result.html`.
- User inputs (patient and query type) were handled using the Flask route function `index()` in `app.py`, which correctly captured form data and redirected requests for processing.

- The reasoning workflow was evaluated using the Flask function `result_page()`, which:
  - Invoked the ASP solver using `subprocess.run()`.
  - Passed the files `generated_ontology.lp` and `reasoning_rules.lp` to Clingo.
  - Ensured successful execution by checking the solver's return code and output.
- The correctness of symbolic reasoning was verified by examining the ASP output parsed using `json.loads()` inside `result_page()`. The extracted answer sets matched the expected medical inferences.
- The helper function `prettify(atom)` was evaluated by checking whether internal ASP predicates (e.g., `patienthassymptom/2`, `recommendtreatment/2`) were correctly converted into human-readable medical statements.
- The RDF to ASP conversion process was evaluated using the script `rdf-to-asp.py`, ensuring that:
  - Classes were extracted correctly.
  - Individuals and relationships were translated into ASP facts.
  - The output file `generated_ontology.lp` was generated without errors.
- User interface usability was evaluated by verifying that:
  - `index.html` correctly displayed patient and query options.
  - `result.html` dynamically rendered reasoning results passed from Flask using Jinja templates.
- Error handling was tested by triggering invalid or empty outputs, confirming that exception handling blocks in `result_page()` (e.g., `TimeoutExpired`, `JSONDecodeError`) responded with appropriate messages.
- Performance evaluation showed that Clingo execution completed within acceptable time limits for the given dataset, confirming efficient reasoning for small knowledge graphs.
- Overall evaluation confirms that all functions and files interact correctly to provide accurate, explainable, and consistent reasoning results.
- This is a new sentence

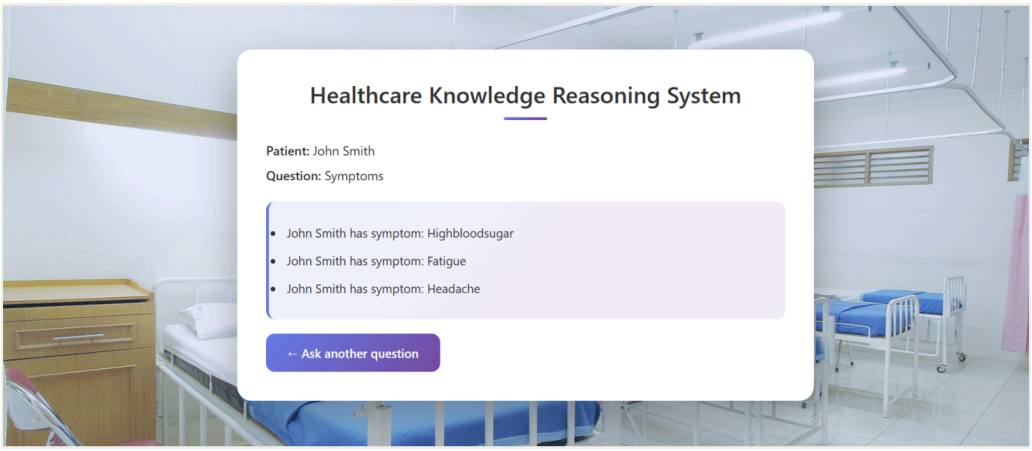

Figure 3: Sample 1: Patient :- John Smith , Question :- Symptoms

4 Discussion and Conclusion

The Healthcare Knowledge Reasoning System demonstrates how semantic web technologies and Answer Set Programming can be effectively integrated to build intelligent clinical decision support tools. By converting RDF/OWL ontologies into ASP format and leveraging Clingo's reasoning engine, the system provides accurate and contextually relevant medical insights including symptom analysis, treatment recommendations, risk assessment, and medication safety checking. The application offers a lightweight backend built using Flask, enabling seamless interaction between the user interface and the ASP reasoning engine through the Clingo Python API.

The web-based interface provides an intuitive platform for healthcare professionals to query patient information and receive reasoned medical insights without requiring technical expertise. The modern, gradient-styled UI with smooth animations enhances user experience, making complex AI reasoning accessible through simple dropdown selections for patients and query types. Results are presented in a clear, well-formatted layout that translates logical inferences into actionable clinical information. While challenges like limited knowledge base coverage and the need for temporal reasoning persist, the project highlights the potential of combining semantic technologies with logic programming as a foundational approach for explainable AI in healthcare.

This project provided valuable hands-on experience in RDF/OWL ontology modeling, Answer Set Programming, and web application development, showcasing how hybrid AI approaches can improve clinical decision-making while maintaining transparency and log-

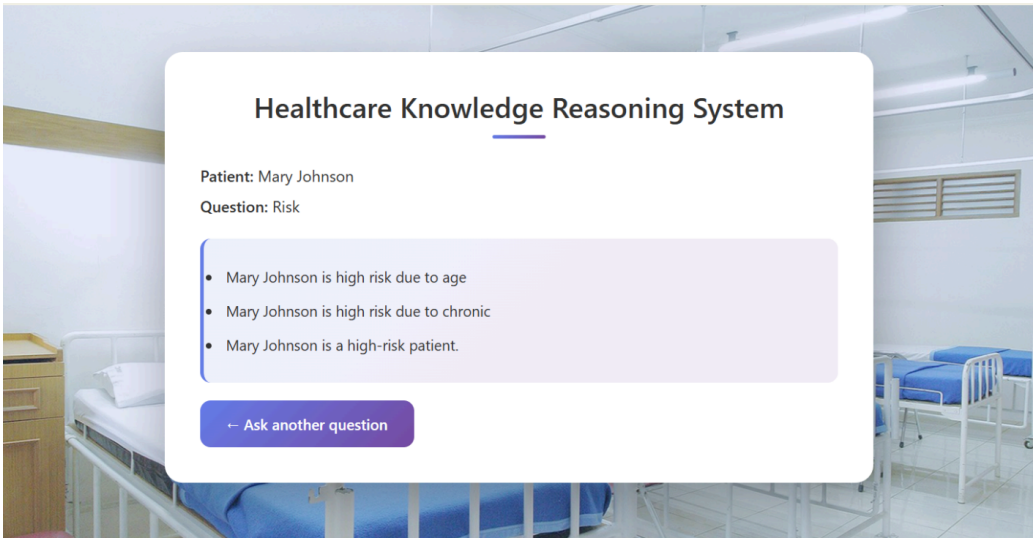

Figure 4: Sample 2: Patient :- Mary Johnson , Question :- Risk

ical consistency. Future enhancements, including integration with standardized medical terminologies like SNOMED CT, probabilistic reasoning capabilities, and clinical validation studies, could elevate the system's performance and usability, making it a competitive solution for modern healthcare decision support needs.

References

[1] D. Allemang, J. Hendler, and F. Gandon, *Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL*. Morgan Kaufmann, 3rd ed., 2020.

[2] C. Baral, *Knowledge Representation, Reasoning and Declarative Problem Solving*. Cambridge University Press, 2003.

[3] M. Gebser, B. Kaufmann, and T. Schaub, *Answer Set Solving in Practice*. Morgan & Claypool, 2012. https://potassco.org/book/.

[4] Stanford University, *Protégé Ontology Editor*. Stanford Center for Biomedical Informatics Research, 2024.

[5] Potassco Team, *Clingo: A Grounder and Solver for Logic Programs*. University of Potsdam, 2024. https://potassco.org/.